SEQFUZZ:
Fuzzing the PHP interpreter via a Sequence-Based Intermediate Representation

***Abstract*—PHP is one of the most widely used server-side programming languages, yet the security of its interpreter remains underexplored. In this paper, we first present SEQIR, a sequence-based intermediate representation that explicitly encodes both control flow and data flow, enabling robust and fine-grained program mutations. Then building on SEQIR, we design SEQFUZZ, a fuzzing framework that translates PHP programs into hierarchical sequences, performs semantically aware mutations, and lifts the mutated sequences back to executable PHP. Our evaluation on the official PHP interpreter demonstrates that SEQFUZZ discovers 65 confirmed bugs, including 52 zero-days, achieves higher code coverage than state-of-the-art fuzzers, and produces a higher proportion of valid seeds with high throughput. Ablation studies show that both control flow and data flow mutation operators are essential to performance. These results highlight the importance of expressive program representations and semantically informed mutations in advancing interpreter fuzzing.**

1. Introduction

The PHP interpreter is the runtime that powers over 70% of all websites, including platforms such as WordPress and Drupal [1], [2]. It is also a memory-unsafe attack surface as the official interpreter is implemented in millions of lines of C, and 87% (133/152) of all PHP CVEs assigned in the past decade are related to memory-corruption bugs inside the interpreter [3]. Exploitation of these bugs has produced denial-of-service, information-disclosure, and arbitrary code-execution outcomes in production deployments [4], [5], [6]. Despite this, the interpreter itself has received far less security attention than the PHP applications layered on top of it.

Yet, finding vulnerabilities in interpreters remains a difficult task. Interpreters accept highly structured inputs and exhibit stateful, deeply semantic behaviors. A fuzzer must produce inputs that satisfy strict syntactic constraints *and* exercise subtler semantic features that touch the interpreter's internal state.

Existing PHP-specific fuzzers fall into two categories, and both miss a large class of interpreter bugs. FlowFusion [7], the official fuzzer used by the PHP development team, splices data-flow subgraphs across seeds but ignores control structure, limiting its ability to explore deeper execution behaviors dependent on control flow such as optimization. Domato [8] and PolyGlot [9] are grammar-driven generators, so they produce syntactically valid PHP but track no semantic dependencies between statements, and their grammars age out as the interpreter evolves. Fuzzing IRs developed for other systems (e.g., JavaScript) have demonstrated that exposing semantic structure improves mutation quality; however, because these IRs target a different language their structural constraints limit their applicability for PHP [10], [11], [12].

Insight. Uncovering a diverse set of interpreter bugs in the PHP interpreter requires a rich representation of both control flow and data flow. Mutating both, at low cost and without breaking program structural validity, requires a representation in which both flows are first-class but uncoupled, and in which a broad class of mutations preserves validity by structure rather than by post-mutation repair.

Approach. We design SEQIR, a fuzzing-oriented intermediate representation for PHP that encodes programs as partially-ordered, hierarchical sequences of dependency-annotated control and data nodes. (1) SEQIR makes joint control- and data-flow mutation cheap because reordering, inserting, deleting, and splicing nodes preserve well-formedness by construction; (2) scope is encoded by sequence nesting rather than by a separate scope graph; and (3) dependencies are stored as references to node objects so lifting re-emits referenced nodes automatically. These properties collectively eliminate the post-mutation processing machinery prior fuzzing IRs require for mutation validity.

Results. We instantiate SEQIR in SEQFUZZ, a fuzzer for the official PHP interpreter that translates seeds from PHP into SEQIR, mutates the resulting sequences with control- and data-flow operators, and lifts mutated sequences back to executable PHP.

We have implemented and evaluated SEQFUZZ on the official PHP interpreter. Over one month of fuzzing, SEQFUZZ found **65 bugs**, including **52 zero-days**. Empirically, SEQFUZZ achieves higher code coverage and finds more unique bugs than state-of-the-art fuzzers, while also generating more semantically valid programs. An ablation study further shows that combining both control flow and data flow mutations is critical to SEQFUZZ's performance.

Contributions. This paper makes the following contributions:

- **SEQIR: A Sequence-based IR for PHP.** A novel IR that encodes both control flow and data flow semantics explicitly, enabling expressive program transformations.

Listing (1) Input Seed

```
1 $a = 0;
2 $b = 1 + 2;
3 $c = 3;
4 $d = $c + 4;
```

Listing (2) Mutated Seed

```
1 $a = 14;
2 $b = (1 + 2);
3 $z = 3;
4 $d = $z + 4;
```

Figure 1: Example of AST mutations. On Line 1, `$a` is mutated despite being unused, producing no behavioral change. On Line 2, a `BinaryOpPlus` node is wrapped in an `Expr` node, introducing redundant parentheses without altering execution. On Line 3, `$c` is renamed to `$z`, but program semantics remain unchanged.

- **A Fuzzing Framework Based on Semantically-Aware Mutation.** A mutation engine built to operate on SEQIR that performs semantically safe, fine-grained transformations to explore deep interpreter logic.
- **Extensive Empirical Evaluation on the Official PHP Interpreter.** A thorough evaluation demonstrating improved coverage and the discovery of numerous previously unknown bugs compared to state-of-the-art fuzzers.

2. Background

The PHP Interpreter. The PHP interpreter is implemented in millions of lines of memory-unsafe C and composed of three primary interacting subsystems: the Zend engine (*zend* directory), the core modules (*ext* directory), and the main functions (*main* directory) [7]. The Zend engine is the central execution engine, translating PHP scripts into intermediate opcodes that are executed by the interpreter. The core modules are responsible for functionality beyond the main interpreter like regular expression parsing. The main functions handle PHP runtime code and are responsible for initializing and executing the interpreter. The interpreter further supports advanced features such as a Just-In-Time (JIT) compiler, which detects and optimizes frequently executed code paths.

The combination of a complex architecture and a memory-unsafe implementation language exposes a broad attack surface. An analysis of historical vulnerabilities shows that **87% (133/152)** of all PHP CVEs over the last ten years stem from memory corruption bugs within the interpreter [3]. Exploitation of these bugs can lead to denial-of-service (DoS) [4], information disclosure [5], or arbitrary code execution [6]. The complex nature of the PHP interpreter and the persistence of high-impact vulnerabilities make the PHP interpreter a compelling target for security research.

Existing PHP Representations. A fuzzer's underlying representation constrains both the types of mutations that are possible and the granularity at which those mutations occur. A number of PHP representations have been developed but are lacking in expressiveness and robustness.

FlowFusion [7], the state-of-the-art PHP interpreter fuzzer, operates at statement-level granularity: each statement is a node annotated with data flow information. FlowFusion applies regular expressions to extract variables from seeds and merges their data flows to generate new test cases. While this approach enables semantically meaningful data flow mutations, it is agnostic to control flow, limiting its ability to explore deep execution behaviors.

UChecker [13] proposed a PHP representation called the heap graph. By symbolically interpreting the AST of PHP programs, heap graphs extract the data flow of PHP programs. The symbolic nature of heap graphs removes control structures such as loops and syntactic elements like functions. Furthermore, similar to other symbolic analysis strategies, heap graphs suffer from path explosion thus limiting their effectiveness in representing more complex PHP programs.

To support JIT compilation, the PHP interpreter uses a graph-based IR [14]. Program dependence graphs like PHP's JIT IR explicitly encode both data flow and control flow [15]. Yet, while optimization IRs preserve the semantic equivalence of the original program, optimization is often a destructive process. In the case of PHP, JIT IR cannot be retranslated into PHP code, eliminating its utility for fuzzing.

In addition to its JIT IR, the PHP interpreter constructs an abstract syntax tree for each script [16]. Prior work on AST-based fuzzers for dynamic languages such as JavaScript [17] shows that ASTs can serve as mutation targets. However, while ASTs implicitly capture high-level semantic constructs, they do not provide an explicit or easily manipulable representation of control flow or data flow. Instead, ASTs primarily encode syntax, which often leads to mutations that are syntactically valid but semantically ineffective [15]; such mutations waste fuzzing cycles. Figure 1 highlights three such cases.

IRs for Dynamic Languages. Prior work on language fuzzing has produced three representative IR designs.

FuzzIL [10], used by Fuzzilli, is a bytecode-level IR that encodes JavaScript as a flat, linear sequence of instructions in single static assignment (SSA) form, where control structure is expressed through paired `Begin`/`End` block markers. However, because FuzzIL is represented as bytecode, data flow and control flow are not explicitly expressed.

CodeAlchemist [11] decomposes JavaScript programs into statement-level fragments (*code bricks*) with assembly constraints describing what context every fragment requires. Mutation proceeds by recombining fragments whose constraints are mutually satisfiable. While the assembly constraints of CodeAlchemist reduce semantic errors, CodeAlchemist operates at statement-level granularity and is thus unable to perform the fine-grained mutations necessary to efficiently find bugs such as integer overflows.

FlowIR [12] represents JavaScript programs as a coupled graph: a control flow subgraph (CFG) and a data-dependency subgraph (DDG) sharing nodes. This exposes both control and data flow explicitly. Mutation operates by graph rewriting: inserting a control construct entails edge insertion, deleting a node entails edge cleanup, and the IR's lifter performs a scope-reconstruction analysis pass to determine variable visibility in the regenerated code. Further-

more, a stated limitation of FlowIR is its generation through intra-procedural analysis. This narrow analysis prevents FlowIR from modeling a program's global context and inter-procedural interactions. SEQIR addresses both limitations.

Critically, PHP contains specific features which are unique to itself. Features such as variable variables (e.g., `$$var`) and reference operators (e.g., `&var`) are not represented by these pre-existing works.

3. Threat Model

We organize attacker capability into three tiers. **T1 (arbitrary PHP source):** the attacker supplies syntactically valid PHP that the interpreter parses and executes; realistic settings include multi-tenant hosts, CMS plugin or theme upload, and sandbox-escape configurations where `disable_functions` or `open_basedir` is treated as a security boundary. **T2 (attacker-controlled `unserialize()` payload):** the byte string reaching `unserialize()` in a PHP application, the classical PHP exploit primitive behind POP chains. **T3 (attacker-controlled structured non-PHP input via PHP APIs):** bytes flowing into `SoapServer`, `DOMDocument::loadXML`, `XMLReader::open`,and similar entry points.

Bugs that SEQFUZZ discovers in `soap`, `dom`, `xmlreader`, `xml`, `phar`, `gd`, `openssl`, and `mbstring` are plausibly T3-reachable; bugs in other components (e.g., Zend engine, JIT, and `streams`, etc.) typically require T1 absent a constructed primitive. Of the 28 release-build crashes, 7 are use-after-free in heap-allocated PHP objects, a class for which interpreter-exploit primitives have historically been demonstrated [9]; we do not construct such primitives in this paper. The immediate consequences supported by our findings are denial-of-service (the 28 release-build crashes) and resource exhaustion in long-running PHP-FPM workers (the 13 CWE-401 memory leaks). Out of scope: kernel-level access, supply-chain compromise of the PHP source tree, privileged configuration changes (e.g., `php.ini` as root), and side-channel attacks against the host CPU.

4. SEQIR

This section describes the design of SEQIR, our sequence-based intermediate representation for PHP. We begin by identifying the challenges in developing a fuzzing-oriented PHP IR. These challenges are addressed by our key insights which are organized into SEQIR.

4.1. Challenges

Program representations are the most crucial part of interpreter fuzzing as they determine the set of mutation operators. However, the shortcomings of currently available IRs reveal the unique challenges presented by fuzzing-driven IR design for PHP. Table 1 summarizes the following challenges and current IRs' abilities to address them.

TABLE 1: Comparison of interpreter IRs across capability properties

Representation	Year	Data Flow	Control Flow	Syntax	Recompilable	Inter-procedural	Scoping	Dependencies	Global Context
(PHP)									
Data Flow Nodes [7]	2025	●	·	●	●	●	·	·	●
Heap Graphs [13]	2019	●	·	·	●	●	◗	●	●
JIT IR [14]	2023	●	●	·	·	·	·	●	·
Abstract Syntax Tree [16]	2014	◗	◗	●	●	◗	◗	·	●
(Non-PHP)									
FuzzIL [10]	2023	◗	◗	·	●	●	●	●	●
Code Bricks [11]	2019	●	·	●	●	●	·	●	●
FlowIR [12]	2024	●	●	●	●	·	●	◗	·
SEQIR	2026	●	●	●	●	●	●	●	●

● Fully supported, explicit feature.
◗ Partially supported, implicit feature (e.g., abstract syntax tree supports data and control flow but data is implicitly contained within the structure).

Data Flow. Altering the data flow of a program tests an interpreter's ability to handle unexpected inputs, yet data flow is not immediately clear from the source code level. An inability to clearly represent the data flow of a program can lead to mutations on inconsequential elements.

Control Flow. Given PHP's usage of JIT compilation and code optimization techniques, an IR must express control flow. Control flow modeling requires proper variable scoping. Additionally, design choices can lead to path explosion problems, rendering the IR unscalable.

Syntactical Expressiveness. A fuzzing-oriented IR must express the diverse syntax usages of a language. If syntactical features are stripped and the original source code reduced to a pure semantic representation of a program, the fuzzer will not be able to test unique areas of the PHP interpreter.

Recompilability. To test the interpreter, the IR must be recompiled into PHP source code. This recompilation process must not introduce syntactical errors and must successfully represent the IR mutations back into the original language.

Scope. A fuzzing-oriented IR must make lexical scope explicit so that mutations do not introduce undefined variables. In representations where scope is implicit, determining which values are visible at a given program point requires non-local analysis. Without an explicit notion of scope, the fuzzer must either repeatedly recompute visibility or generate seeds that fail trivially, wasting fuzzing cycles before any interpreter logic is exercised.

Dependencies. A fuzzing-oriented IR must record the dependencies of each program element so that mutations can be applied freely without breaking semantic integrity. When dependencies are implicit, the fuzzer cannot tell which elements a given node relies upon, forcing it to either restrict mutations to a narrow, order-preserving set or re-analyze the program after mutation to recover the necessary relationships.

Inter-Procedural Representation. Many interpreter behaviors, such as inlining and closure semantics, depend on interactions across procedural boundaries. A representation that confines analysis to a single procedure cannot express inter-

procedural mutations such as splicing a control structure from one function into another.

Global Context. Mutation operators frequently need access to program-wide state: which variables are in scope, which functions are defined, which classes exist. Representations that cannot represent this global state are limited in their capacity to express diverse programs.

4.2. Insights

To address these challenges, SEQIR introduces novel design properties to advance the state-of-the-art in PHP program representation. These invariants hold by construction as they are preserved by the structure of SEQIR rather than enforcement logic. These innovations form the basis for our system design.

- **Partial-ordering.** To maintain structural integrity, SEQIR imposes a single structural constraint: each subsequence begins with a control node. This light constraint is sufficient to capture a program's control flow and allows for flexible mutations.
- **Annotated Nodes.** To represent data and control elements, SEQIR uses symbolic nodes. Nodes are annotated with their data and structural dependencies, making each node independently liftable to a semantically correct PHP statement and enabling mutations such as splicing.
- **Hierarchical Sequences.** Because each node represents valid statements, nodes can simply be sequentially arranged. The sequential structuring avoids the problem of maintaining tree integrity while the hierarchical arrangement shows clear variable scoping.
- **Syntax Preservation.** Instead of using a purely symbolic approach, SEQIR preserves syntactical elements, enabling the testing of PHP's rich syntax and features.

4.3. Detailed Design

Our primary contribution, SEQIR, is a semantically expressive, mutation-resilient IR that makes both control flow and data flow between PHP constructs explicit while preserving a structure suitable for fine-grained fuzzing. This representation underlies the diverse mutation operators implemented in SEQFUZZ. At its core, SEQIR models programs as partially-ordered, hierarchical sequences of elements rather than as trees or graphs. This sequence-based structure provides a flexible, unambiguous encoding of scope, flow, and dependency.

Each element in a sequence is either a **data node** or a nested **subsequence** beginning with a **control node** (§4.3.1). Unlike traditional graph-based program models, where lexical scope is implicit and determining which values are visible within a region may require non-local analysis, SEQIR encodes scope structurally. By organizing nodes into nested sequences, the scope of each subsequence is explicit. Data nodes introduced in a sequence are visible to all of its subsequences and their descendants (§4.3.2). This structural

Listing 3: PHP to SEQIR

```
function foo($a) {
        $y = 2;
        $x = 14;
        $z = $a ^ $x;
        return $z;}
 $a = 1;
 $b = 8;
if($a > 1) {
        $a = foo($a);
} else {
        $b = 4 / $a;}
```

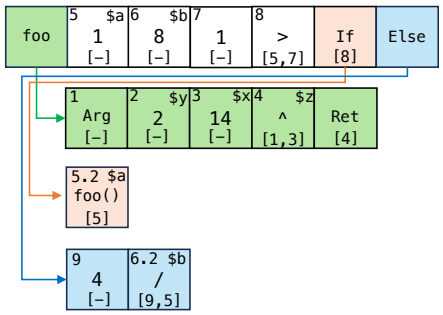


Figure 2: SEQIR of Listing 3. Boxes with exiting arrows denote control nodes, while all other nodes denote data nodes. Node labels appear in the upper-left corner of each box with relevant source-code variable names in the upper-right. Dependencies appear in brackets at the bottom. The uppermost row is the main sequence, and control nodes point to their subsequences. Each sequence is color-coded to its associated source code region.

encoding replaces implicit scoping heuristics with a clear, hierarchical model.

Formally, a SEQIR sequence S is defined as:

$$S = [\, S_{\text{sub}0} \text{ or } D_0,\ S_{\text{sub}1} \text{ or } D_1,\ \ldots,\ S_{\text{sub}n} \text{ or } D_n \,].$$

Each $S_{\text{sub}i}$ denotes a nested subsequence of the form:

$$S_{\text{sub}} = [\, C,\ S_{\text{sub}0} \text{ or } D_0,\ S_{\text{sub}1} \text{ or } D_1,\ \ldots,\ S_{\text{sub}n} \text{ or } D_n \,],$$

- C is a control node (e.g., `If`, `For`, `Try`, `Function`).
- Each D_i is an unordered collection of data nodes accessible in the current lexical scope.

Control nodes represent structural and ordering constraints derived from the program's control flow. Data nodes represent individual data operations, each annotated with dependency information indicating which prior nodes determine its value. These annotations make both intra- and inter-sequence dependencies explicit, enabling reasoning about data flow without requiring a separate def-use graph.

Because nodes are annotated with dependencies, nodes can be freely positioned within a sequence (with the exception of the starting position being a control node). De-

pendencies are automatically resolved when SEQIR is lifted back to PHP by evaluating annotations (Algorithm 2). This partial-order formulation makes SEQIR highly mutable. Mutation operators may reorder, insert, delete, or splice nodes because annotations maintain dependency constraints. This combination of explicit control structure, explicit data flow dependencies, and partially ordered sequencing provides SEQFUZZ a semantically coherent model that offers substantial flexibility for generating program variants.

4.3.1. Nodes. SEQIR sequences are constructed from data nodes and control nodes with each type capturing a specific aspect of program behavior. These nodes make control flow and data flow explicit, allowing SEQFUZZ's mutation operators to reason about program semantics and apply mutations that meaningfully alter execution rather than merely changing syntax.

Data Nodes. Data nodes represent the computational units of SEQIR and correspond to constant values or to the results of applying an operator. Operators range from arithmetic operations to object- and array-oriented constructs such as function calls and array indexing. Therefore, each data node encapsulates a single semantic action in the program.

To maintain interpretability, every data node stores the value it produces, which can be mapped directly back to its source-level representation. For example, Node 5 in Figure 2 encodes the constant `1`, originating from Line 6 of Listing 3. This traceability ensures that SEQIR retains a faithful semantic correspondence to the original PHP program.

SEQIR abstracts user-defined variable names and assigns data nodes unique identifiers. Each identifier denotes a single definition, ensuring that data flow is unambiguous and making def–use chains explicit. In Figure 2, Node 5 represents the initial assignment on Line 6 of Listing 3, while Node 5.2 represents the subsequent reassignment on Line 9. These multiple definitions are versions in SEQIR are simply reduced into a single source-level variable during lifting as outlined in §5.3.

Additionally, each data node records its dependencies. In this context, there are two dependency types. Data dependencies represent the values a node consumes as operands. Structural dependencies capture the required pre-existing structure for a node to be valid (e.g., function definitions). Explicit dependency tracking is vital for mutation as it prevents the fuzzer from constructing syntactically valid but structurally impossible programs (e.g., calling an undefined function).

Control Nodes. Control nodes encode constructs that alter the execution flow of a program. These include conditional branches, loops, function and method entry points, and other constructs that introduce new control contexts. Unlike data nodes, a control node does not compute a value; instead, it delineates which subsequences of nodes may execute and establishes the hierarchical structure of a SEQIR program.

As with data nodes, each control node is annotated with its dependencies. Most control dependencies arise from values that determine execution flow (e.g., the Boolean value of an `if` statement). The remaining dependencies are other control nodes (e.g., an `Else` node is dependent on an `If` node). By explicitly recording these dependencies, SEQIR preserves the semantic constraints necessary to mutate control structures safely. During transformation, mutations that alter or relocate control nodes remain well-formed because the interpreter's required ordering and value conditions are preserved.

In Listing 3, the program yields three control nodes in Figure 2: a `foo` function-entry node, an `If` node, and its paired `Else` node. The `If` node is annotated with a dependency on the Boolean value computed by Node 8, ensuring that the branch condition remains semantically linked to its controlling expression even if the surrounding sequence is mutated.

4.3.2. Main Sequence and Subsequences. SEQIR represents programs as hierarchical, partially ordered sequences. This structure offers three advantages over graph- or tree-based IRs: clear scoping, minimal constraints, and simplified manipulation.

Scoping. In SEQIR, hierarchical sequences expose lexical scoping directly; a sequence corresponds directly to a scope, and its nested subsequences correspond to nested scopes. The *main sequence* represents top-level execution and imposes no restrictions on its first node. Each *subsequence*, in contrast, begins with a control node that defines the branching behavior that governs its execution. Subsequences may nest arbitrarily. Each sequence, with the exception of isolated procedures (e.g., functions, classes), inherits all variables visible in its parent sequence, preserving PHP's lexical scope rules.

Minimal Constraints. The only ordering constraint in SEQIR is that a subsequence must begin with a control node. This minimal restriction is sufficient to preserve the original program's control structure. It also enables mutations to introduce or remove nodes and subsequences at almost any position without violating structural validity.

Simplified Manipulation. The sequential structure of SEQIR simplifies the mutation process. Unlike tree-based IRs where mutations risk breaking structural or semantic constraints, SEQIR attaches semantic information directly to its nodes. Because dependencies are explicit, operations such as splicing or reordering preserve semantic integrity automatically.

Listing 3 produces four sequences shown in Figure 2. The main sequence contains three subsequences: `foo`, `If`, and `Else`. Each subsequence contains its associated data nodes and starts with a control node describing its behavior.

5. SEQFUZZ

This section describes the detailed design of SEQFUZZ, our prototype fuzzer that performs structured program mutation on SEQIR (§4). Figure 3 presents a high-level overview of SEQFUZZ. SEQFUZZ consists of three primary modules:

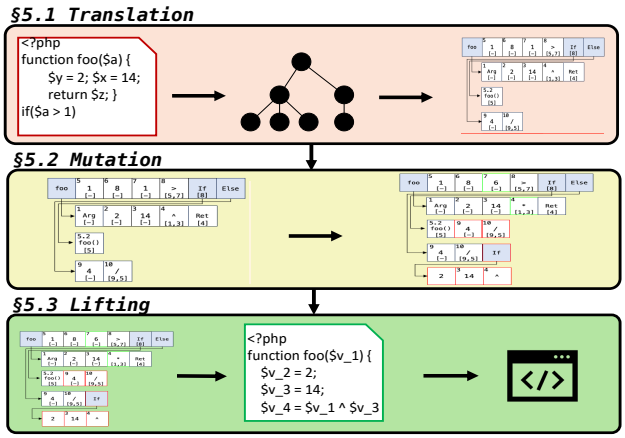


Figure 3: Overview of SEQFUZZ

Algorithm 1: Translation of PHP AST into SEQIR

```
   Input: Abstract Syntax Tree (AST)
   Output: SEQIR sequence
1  Function Translate(AST):
2      Sequence ← [ ]
3      foreach node ∈ AST do
4          EvalNode (node, Sequence)
5      end
6      return Sequence
7  Function EvalNode(node, Sequence):
8      if GetNodeType(node) = Expr_BinaryOp then
9          left ← EvalNode (node.left, Sequence)
10         right ← EvalNode (node.right, Sequence)
11         IRNode ← SeqDataNode(left, right, node.value)
12         Sequence.append(IRNode)
13         return IRNode
14     else if GetNodeType(node) = Expr_If then
15         LocalSeq ← [ ]
16         Cond ← EvalNode (node.cond, Sequence)
17         IRNode ← SeqControlNode(Cond)
18         LocalSeq.append(IRNode)
19         foreach SNode ∈ node.stmts do
20             EvalNode (SNode, LocalSeq)
21         end
22         return LocalSeq
23     else
           // Other node types handled similarly
24     end
```

1) **Translation (§5.1).** As regression tests are designed to exercise diverse interpreter functionality, they provide a natural seed corpus. SEQFUZZ extracts tests from the official PHP suite [18] and translates each seed into SEQIR.
2) **Mutation (§5.2).** To explore interpreter behaviors, SEQFUZZ mutates SEQIR by transforming both control flow and data flow structures. These mutations target semantic changes rather than syntactic alterations, increasing the likelihood of reaching unexplored interpreter states.
3) **Lifting (§5.3).** Mutated SEQIR programs are lifted back to PHP source code, enabling seeds with transformed control and data flows to be executed on an instrumented PHP engine for crash detection.

5.1. Translation

To take advantage of existing seed corpora, SEQFUZZ supports translation of PHP programs into SEQIR. Regression suites are widely recognized as high-quality starting points for fuzzers [15], [7]. The PHP development team maintains over 20k test scripts designed to validate interpreter functionality. Translating this suite into SEQIR enables SEQFUZZ to reuse these realistic programs as initial seeds, providing broad coverage of language constructs and execution behaviors.

SEQFUZZ translates PHP programs into SEQIR by lowering their AST representation (Algorithm 1). As shown in the function `EvalNode` on Line 7, translation performs a post-order traversal of the AST and recursively converts each AST node into an appropriate SEQIR construct. Expression nodes (e.g., binary operations) become data nodes that explicitly encode operators and operands (Line 8), while control flow constructs (e.g., conditionals) initiate subsequences rooted at control nodes (Line 14). Throughout translation, SEQFUZZ records references among SEQIR nodes to maintain both data and structural dependencies. The resulting SEQIR program is thus a faithful and analyzable representation of the original PHP seed. Figure 2 shows an example translation.

5.2. Mutation

SEQIR mutations fall into two categories. (1) Data flow mutations modify the values or operands consumed by operators, enabling exploration of alternative value computations. (2) Control flow mutations alter the program's execution structure. Together, they allow SEQFUZZ to systematically explore both value-level and structural variations within seeds, significantly expanding the reachable program space while maintaining semantic coherence.

5.2.1. Data Flow Mutations. Data flow mutations alter the values consumed by operators or the operators themselves. These mutations allow SEQFUZZ to explore a broad range of value-driven behaviors, ranging from subtle runtime conversions to extreme edge cases. Because each data node is annotated with explicit data and structural dependencies, data flow manipulations can be performed without violating variable scope or operand availability.

Data Flow Splicing. SEQIR's explicit dependency model allows data flow edges from distinct programs to be recombined directly. Given two seeds, SEQFUZZ can splice their data flows by substituting any data node with another data node. Crucially, since dependencies are recorded, substitution never introduces dangling references. For example, a node producing the constant `42` may be replaced with a node representing the result of a function call. When doing so, all required dependencies are automatically carried over because each node includes its structural prerequisites.

This splicing mechanism creates hybrid data flow graphs that do not appear in the original corpus, allowing SEQFUZZ to explore data-driven paths through the interpreter

Listing 4: PHP to splice

```
function bar(){
    $q = 15;
    $s = 18;
    $z = $q + $s;                    // DF Snippet
    return $z;}
-------------------------------------------
$i = array();
foreach($i as $item) {               // CF Snippet
    $j = $item * 20;}
```

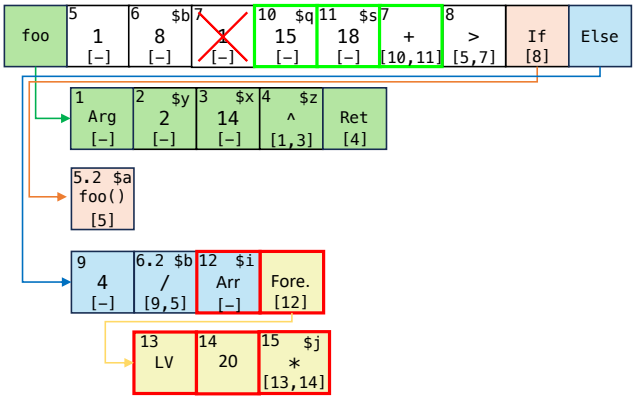


Figure 4: Splicing control and data flows into Listing 4. Green boxes correspond with the data flow snippet while red boxes represent the control flow snippet. The data flow snippet is spliced into the old Node 7 (red X), thus all data nodes utilizing Node 7 use the new data flow. The control flow snippet is spliced into the `Else` statement, producing a new, nested control sequence.

that would otherwise be unreachable. Since PHP's dynamic semantics often hinge on runtime values, data flow recombination is a powerful driver of differential execution behavior.

Operator and Operand Replacement. To expand the semantic variability of generated seeds, SEQFUZZ performs targeted rewrites of operators and operands. For operator replacement, the operator associated with a data node may be substituted with another operator of the same arity. For instance, an arithmetic operator + may be replaced with /, while a comparison operator such as == may be replaced with <. Operand replacement similarly enables value diversification with minimal structural disruption. For example, a constant `1` can be replaced with `PHP_INT_MAX`.

5.2.2. Control Flow Mutations. Control flow mutations alter the structural ordering and branching between statements. With PHP's support of optimization and JIT compilation, control flow mutations are essential for exercising these capabilities. Because SEQIR models control constructs as hierarchical sequences rooted at control nodes, SEQFUZZ can manipulate control flow while preserving program integrity.

Control Flow Splicing. SEQIR's sequence-based representation allows control structures to be transplanted between

Listing 5: Figure 4 lifted to PHP

```
function foo($v_1) {
    $v_2 = 2;
    $v_3 = 14;
    $v_4 = $v_1 ^ $v_3;
    return $v_4;
}
$v_5 = 1;
$v_6 = 8;
$v_10 = 15;                          // Spliced DF
$v_11 = 18;                          //     ...
$v_7 = $v_10 + $v_11;                //     ...
$v_8 = $v_5 > $v_7;
if($v_8) {
    $v_5 = foo($v_5);
} else {
    $v_9 = 4;
    $v_6 = $v_9 / $v_5;
    $v_12 = array();                 // Spliced CF
    foreach($v_12 as $LV) {          //     ...
        $v_14 = 20;                  //     ...
        $v_16 = $LV * $v_14; }       //     ...
}
```

seeds with minimal constraints. A subsequence headed by a control node can be inserted into any location in another sequence. Given node annotations, all dependencies are automatically resolved, thus inserting a control structure does not introduce undefined identifiers or broken references.

For instance, an `if`-statement extracted from one seed may be embedded inside a loop body from another, creating a nested control structure that induces new branching patterns. This cross-seed recombination forces the interpreter to navigate execution paths that do not appear in the original corpus, increasing the likelihood of exercising rarely tested sections associated with branching, scoping, or stack management.

Condition Replacement. The condition expression associated with a control node can be replaced with another data node. This mutation preserves the control flow structure while altering the runtime criteria that govern branch selection. Replacing a guard such as `($x > 0)` with an expression like `($y == 42)` changes which subsequence executes without affecting the syntactic structure of the program.

Condition Body Replacement. SEQIR further enables structural mutations by replacing the body of a control construct with another subsequence. For example, the statement block inside an `if`-branch can be replaced with the loop body of a `while`-construct from a different seed. Although the resulting pairing may appear semantically unusual, it remains syntactically valid because subsequences are self-contained and dependencies are preserved. This type of

mutation forces the interpreter to evaluate unanticipated control combinations.

5.3. Lifting

Lifting mutated SEQIR back into executable PHP is a critical step, as the interpreter ultimately operates on source-level programs (Algorithm 2). The lifting process must reconstitute PHP code that (1) faithfully reflects the data flow and control flow mutations applied at the IR level, (2) preserves the syntactic richness of the original programs, and (3) respects lexical scoping to avoid introducing undefined variables or structural inconsistencies. Lifted code is then executed to detect interpreter bugs. Listing 5 presents SEQIR being lifted to PHP source code.

Reconstruction of Control and Data Flow. SEQFUZZ lifts SEQIR sequences in a left-to-right, top-down traversal that mirrors the hierarchical organization of SEQIR (Line 8). Control nodes are emitted first, followed by the recursive lifting of their subsequences (Line 20). This enforces the control flow structure encoded in the IR and ensures that nested bodies appear in the correct order in the regenerated PHP code.

Demonstrated on Line 12, data nodes are lifted by recursively emitting the PHP expressions corresponding to their dependencies. Because SEQIR preserves data dependencies, the lifted code maintains the intended evaluation order without requiring further analysis. The lifting process records emitted nodes to avoid duplicate emission when a node is used across multiple expressions (Line 10).

Preservation of High-Level Syntax. Fuzzing effectiveness depends heavily on preserving syntactic diversity and the high-level constructs seen in human-written programs. Rather than lowering syntax into an abstract, uniform set of operations (e.g., heap graphs), SEQIR maintains specialized node types corresponding to rich PHP constructs. For example, `FuncCallNode` encodes function invocations with their original argument structure, and `InterpolatedStringNode` preserves PHP's string interpolation semantics. Similarly, SEQIR includes dedicated control nodes such as `TryCatchNode`, enabling the lifted program to recover the original high-level semantics of the seed. This design ensures that mutations occur at the semantic level while still producing syntactically varied programs.

Scope Preservation. Scoping correctness is essential to producing executables that do not crash trivially. Each SEQIR sequence explicitly tracks the set of variables defined within its lexical region. Subsequence inheritance mirrors PHP's nested scoping rules, ensuring that all variables referenced by a node are defined in an enclosing sequence.

During lifting, variable references are resolved through the unique identifiers recorded in their defining data nodes (Line 18). Because each data node recursively prints its dependencies, the lifted code naturally reconstructs correct scope. This prevents undefined variable errors and guarantees that the lifted program preserves the lexical structure encoded by SEQIR.

Algorithm 2: Lifting SEQIR to PHP

```
   Input: SeqIR sequence
   Output: PHP source string (list of statements)
1  Function Lift(sequence):
2      PHPString ← [ ];
3      Visited ← { };                    // nodes already emitted
4      VarMap ← { };    // SeqIR label → PHP identifier
5      foreach IRNode ∈ sequence do
6          REvalNode(IRNode, PHPString, Visited, VarMap);
7      return PHPString;
8  Function REvalNode(IRNode, PHPString, Visited, VarMap):
9      if IRNode ∈ Visited then
10         return ;                      // avoid duplicate emission
11     Visited.add(IRNode);
12     if GetNodeType(IRNode) = DataNode then
13         foreach dep ∈ IRNode.dependencies do
14             REvalNode(dep, PHPString, Visited, VarMap);
           // Ensure operands have PHP identifiers
15         foreach val ∈ GetProducedValues(IRNode) do
16             if val ∉ VarMap then
17                 VarMap[val] ← SeqIRName();
18         PHPString.append(GetExpression(IRNode, VarMap));
19         return
20     else if GetNodeType(IRNode) = ControlNode then
           // Emit the control node (e.g., if)
21         foreach dep ∈ IRNode.condition_dependencies do
22             REvalNode(dep, PHPString, Visited, VarMap);
23         PHPString.append(GetControlHeader(IRNode, VarMap));
           // Emit subsequence body
24         foreach sub ∈ IRNode.subsequence do
25             REvalNode(sub, PHPString, Visited, VarMap);
26         PHPString.append(GetControlFooter(IRNode));
27         return
28     else
           // Other kinds of nodes (e.g., functions)
29         foreach child ∈ IRNode.children do
30             REvalNode(child, PHPString, Visited, VarMap);
```

6. Evaluation

We implemented SEQFUZZ as a black-box PHP interpreter fuzzer in Python, with approximately 5K lines of code. SEQFUZZ uses the official PHP test suite [18] as its initial corpus, parses each seed with PHP-Parser [16], lowers it into SEQIR through a custom translation pipeline, and serializes translated seeds as *pickle* files [19] for reuse across fuzzing iterations. To detect bugs, we execute generated PHP programs on an ASan/UBSan-instrumented PHP interpreter and record sanitizer reports and interpreter crashes for triage. We provide the source code of SEQIR and the prototype fuzzer SEQFUZZ, available at the anonymous link[1].

In this section, we evaluate SEQFUZZ to assess its ability to uncover vulnerabilities in the PHP interpreter and to understand the contribution of its design choices. Our evaluation answers the following research questions:

- **RQ1:** Can SEQFUZZ discover new bugs in the PHP interpreter?
- **RQ2:** How does each mutation strategy impact the overall performance of SEQFUZZ?
- **RQ3:** How does SEQFUZZ's coverage and bug finding compare to state-of-the-art fuzzers?

1. https://anonymous.4open.science/r/seqfuzz-433B/

- **RQ4:** How effective is SEQFUZZ at generating valid and executable PHP seeds?
- **RQ5:** Does operating over SEQIR introduce significant overhead?

6.1. Experimental Setup

We deployed SEQFUZZ on a dedicated server with 16 CPU cores and 180 GB of RAM running Ubuntu 22.04. We fuzzed the official PHP interpreter (`v8.5.0-dev`), compiled daily from source with sanitizers. Following prior work, we initialized all fuzzing campaigns using the official PHP test suite [18].

Answering RQ1: We evaluated SEQFUZZ on `PHP-8.5.0-dev`, the daily-updated snapshot built directly from the PHP master branch. Using the development release ensures that discovered bugs reflect up-to-date behaviors.

We ran SEQFUZZ over a period of one month, enabling AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) to detect memory safety violations and undefined behavior during execution. Each generated seed was executed on the instrumented interpreter, and sanitizer reports were logged for analysis. We triage each crash to identify the affected interpreter component and determine the underlying root cause. All vulnerabilities were responsibly disclosed through the official PHP security reporting process.

Answering RQ2: We perform an ablation study to quantify the individual contribution of each mutation strategy to overall coverage. Because SEQFUZZ combines both data flow and control flow mutations, isolating their effects is necessary to indicate which semantic mutations are most responsible for exercising different interpreter behaviors.

We construct two ablated variants of SEQFUZZ: (1) a *Data-Flow-Only* variant, which disables control flow mutations; and (2) a *Control-Flow-Only* variant, which disables data flow mutations. Each configuration is executed in a 24-hour fuzzing campaign under identical conditions (limited to 8 cores). We measure code coverage at the full interpreter level and at the granularity of major subsystems described in §2. We compare the coverage achieved by each ablated variant against the full configuration to assess how individual mutation classes influence exploration effectiveness.

Answering RQ3: We conduct a comparative study against state-of-the-art PHP fuzzers. While benchmarks such as Magma [20] and FuzzBench [21] are widely used for evaluating general-purpose fuzzers, no analogous benchmark exists for PHP interpreters. This absence of ground-truth corpora necessitates evaluation on real-world software.

Because the PHP interpreter accepts highly structured input, we exclude general, unstructured fuzzers such as AFL and AFL++, which cannot reliably produce syntactically valid PHP programs and thus contribute negligibly in practice. Among PHP-targeted fuzzers, only a subset is publicly available. Notably, LangFuzz [22] is credited with discovering 18 PHP bugs but remains closed source, preventing its inclusion in our study. Given these constraints, we select three representative and state-of-the-art baselines that are publicly available, can generate syntactically valid PHP scripts, and are explicitly designed for structured fuzzing.

FlowFusion [7], the official fuzzer used by the PHP development team, generates new seeds by merging data flows extracted from existing PHP test cases. We use FlowFusion exactly as recommended by its authors, including their provided configuration and workflow. **PolyGlot** [9], a BNF-based, generative fuzzer capable of producing semantically valid inputs through annotated grammar rules. We configure PolyGlot for PHP following its official documentation and use the PHP test suite for its initial seed corpus. **Domato** [8], a grammar-based fuzzer developed by Google Project Zero, includes a built-in PHP grammar. We use Domato's PHP grammar unchanged for a faithful comparison.

We track coverage and bug discovery of all four systems over 24-hour fuzzing campaigns (limited to 8 cores).[2]

Answering RQ4: We evaluate the ability of SEQFUZZ and baseline fuzzers to generate *valid* PHP programs. Because invalid seeds are discarded by the interpreter before exercising meaningful execution paths, seed validity is a critical factor for effective fuzzing throughput.

We generate 1,000 PHP programs and execute each seed on the uninstrumented PHP interpreter. A seed is considered *valid* if it executes to completion without triggering parse errors, uncaught exceptions, or fatal runtime errors. This definition captures programs that successfully reach interpreter logic beyond front-end parsing and error handling.

By default, Domato generates 1000 PHP statements per program and wraps all generated code within `try-catch` blocks. As `try-catch` blocks are prohibitive to assessing the semantic validity of grammar-based seed generation, we disable this feature for our test.

Answering RQ5: We examine the throughput of SEQFUZZ and baseline fuzzers. Because SEQFUZZ operates on SEQIR, throughput is primarily determined by the efficiency of SEQIR manipulation. This analysis reveals whether SEQIR serves as an efficient fuzzing representation compared to established baselines. We assess throughput by recording the number of seed executions performed over a 24-hour period (8 cores).

Coverage Oracle. Code coverage is a standard metric for evaluating the effectiveness of fuzzing techniques [23]. To measure coverage, we use `gcovr` [24] following the recommended instructions provided by its developers. Coverage is collected at the line level across the entire PHP interpreter as well as within major subsystems. Due to the reduction in execution speed, the coverage oracle is only used for comparison studies.

Statistical Rigor. We followed the best practices outlined by Klees et al. [23], [25]. We performed seven independent runs for each ablation, comparison, validity, and throughput experiment (RQ2-5). We used the two-sided Mann-Whitney U test to evaluate whether the observed differences were statistically significant ($p < 0.05$).

2. v8.5-dev, commit 3f3a266a2bab9fda95096cf4a17156a5dc66e8bb

#	Issue ID	Class	Component	Status	Fix	Crash
1	[redacted]	CWE-199 (1)	streams	Fx	+60 -0	✓
2	[redacted]	CWE-122 (2)	zend_test	Vd		
3	[redacted]	CWE-122	soap	Fx	+72 -5	✓
4	[redacted]	CWE-190 (13)	calendar	Fx	+87 -1	✓
5	[redacted]	CWE-190	spl	Fx	+37 -6	
6	[redacted]	CWE-190	date	Dp		
7	[redacted]	CWE-190	gd	Vd		
8	[redacted]	CWE-190	-	Dp		
9	[redacted]	CWE-190	-	Vd		
10	[redacted]	CWE-190	-	Vd		
11	[redacted]	CWE-190	-	Vd		
12	[redacted]	CWE-190	-	Fx	+46 -0	
13	[redacted]	CWE-190	gmp	Dp		✓
14	[redacted]	CWE-190	Engine	Vd		
15	[redacted]	CWE-190	zip	Fx	+87 -1	✓
16	[redacted]	CWE-190	gd	Vd		✓
17	[redacted]	CWE-401 (13)	readline	Dp		
18	[redacted]	CWE-401	zend_test	Vd		
19	[redacted]	CWE-401	iconv	Vd		
20	[redacted]	CWE-401	xml	Vd		
21	[redacted]	CWE-401	soap	Fx	+21 -1	
22	[redacted]	CWE-401	libxml	Vd		
23	[redacted]	CWE-401	xmlwriter	Vd		
24	[redacted]	CWE-401	dom	Vd		
25	[redacted]	CWE-401	openssl	Vd		✓
26	[redacted]	CWE-401	soap	Vd		
27	[redacted]	CWE-401	xmlwriter	Fx	+46 -0	
28	[redacted]	CWE-401	zip	Fx	+37 -0	
29	[redacted]	CWE-401	xmlreader	Fx	+1 -0	
30	[redacted]	CWE-415 (1)	streams	Fx	+16 -8	✓
31	[redacted]	CWE-416 (7)	mbstring	Fx	+13 -2	✓
32	[redacted]	CWE-416	dba	Fx	+35 -2	✓
33	[redacted]	CWE-416	phar	Fx	+18 -1	✓
34	[redacted]	CWE-416	Optimizer	Fx	+45 -4	✓
35	[redacted]	CWE-416	Engine	Dp		
36	[redacted]	CWE-416	spl	Fx	+60 -28	✓
37	[redacted]	CWE-416	uri	Dp		✓
38	[redacted]	CWE-476 (6)	xmlreader	Dp		✓
39	[redacted]	CWE-476	streams	Dp		✓
40	[redacted]	CWE-476	JIT	Dp		✓
41	[redacted]	CWE-476	uri	Fx	+15 -1	✓
42	[redacted]	CWE-476	dom	Dp		✓
43	[redacted]	CWE-476	bcmath	Fx	+22 -1	✓
44	[redacted]	CWE-617 (1)	uri	Vd		
45	[redacted]	CWE-681 (1)	date	Vd		
46	[redacted]	CWE-682 (1)	gd	Dp		
47	[redacted]	CWE-704 (1)	-	Vd		
48	[redacted]	CWE-758 (1)	-	Fx	+25 -0	✓
49	[redacted]	CWE-787 (1)	bcmath	Vd		
50	[redacted]	CWE-824 (1)	streams	Fx	+37 -6	✓

TABLE 2: The first 50 bugs discovered by SEQFUZZ

6.2. Results

6.2.1. Effectiveness in Bug Discovery. The practical impact of SEQFUZZ is demonstrated by its ability to find bugs in the latest PHP interpreter. Over a one-month fuzzing campaign on the daily updated `PHP-8.5.0-dev`, SEQFUZZ discovered **65 confirmed bugs**. Of these, **52 were zero-days** and were acknowledged by the PHP development team. Further analysis revealed **28 vulnerabilities are capable of crashing a release build of the interpreter**.

Distribution. Table 2 lists the first 50 bugs identified by SEQFUZZ. For anonymity, issue identifiers have been temporarily redacted. The *Component* column specifies the interpreter subsystem affected by the bug, the *Class* column lists the corresponding Common Weakness Enumeration (CWE), and the *Status* column indicates whether the bug is fixed (*Fx*), verified (*Vd*), or a duplicate (*Dp*). The *Fix* column reports the number of lines changed to patch the bug. The *Crash* column flags bugs that caused a fatal crash in a release build of the interpreter, showing vulnerabilities that could be simply used in a denial-of-service (DoS) scenario.

Listing 6: Heap use-after-free in SPL

```
1  $v_15488 = new SplMaxHeap(); $v_15506 = 42;
2  $v_15490 = $v_15488->insert($v_15506);
3  $v_15497 = new SplPriorityQueue();
4  class CustomHeap extends SplMaxHeap {
5      public $flags = 'user_property';
6      public $heap_elements = 'user_property';}
7  $v_15505 = new CustomHeap();
8  $v_15507 = $v_15505->insert($v_15497);
9  $v_15492 = $v_15488->insert($v_15507);
10 $v_15494 = $v_15488->insert($v_15494);
11 $v_15495 = $v_15488->__serialize();
12 $v_3982 = 'fl called\n';
13 $v_15496 = var_dump($v_3982);
14 $v_15508 = $v_15505->__serialize();
15 $v_15500 =$v_15497->insert($v_15500,$v_15508);
```

The discovered vulnerabilities span a wide range of categories and severities. The discovered bugs corresponded to **14 distinct CWEs**[3]. The most common classes were:

- **CWE-190**: Integer Overflow (13) - may result in out-of-bounds accesses or memory corruption.
- **CWE-401**: Memory Leak (13) - can enable resource exhaustion.
- **CWE-416**: Use-after-Free (7) - may trigger crashes or undefined behavior.
- **CWE-476**: Null Pointer Dereference (6) - may lead to interpreter crashes.

These bugs were distributed across **29 unique components** of the PHP interpreter, indicating that SEQFUZZ can effectively explore diverse parts of a large codebase instead of concentrating on isolated error-prone modules. Analyzing these bugs on a release version of the interpreter revealed that **28 vulnerabilities caused a fatal crash**, emphasizing their critical nature in production environments.

Vulnerability Analysis: Heap Use-after-Free in SPL. The standard PHP library (SPL) provides interfaces and classes for common operations such as file handling. A use-after-free vulnerability, presented by Listing 6, appears due to the exposure of engine internals to user applications. SEQFUZZ discovers this bug by creating two `SplMaxHeaps` (Line 1 and Line 7) and an `SplPriorityQueue`. SEQFUZZ then nests them in subsequent statements (Lines 8-10) before `serialize` calls (Line 11 and Line 14). Developers stated, "*Exposing INDIRECTS to [users] is not allowed and can lead to all sorts of wrong behaviour. In this case, it led to UAF bugs.*"

3. The bugs discovered by SEQFUZZ correspond to the following CWEs: CWE-119 Improper Restriction of Operations within Buffer, CWE-122 Heap-based Buffer Overflow, CWE-190 Integer Overflow or Wraparound, CWE-401 Missing Release of Memory, CWE-415 Double Free, CWE-416 Use-after-Free, CWE-476 Null Pointer Dereference, CWE-617 Reachable Assertion, CWE-681 Incorrect Conversion between Numeric Types, CWE-682 Incorrect Calculation, CWE-704 Incorrect Type Conversion, CWE-758 Reliance on Undefined Behavior, CWE-787 Out-of-Bounds Write, CWE-824 Access of Uninitialized Pointer. [26]

Listing 7: Memory leak in SOAP

```
1  $v_5237 = 'irrelevant';
2  $v_5256 = 'fOmM6ZzprOm86czp3Ons6fzqDOoc6jz';
3  $v_5257 = base64_decode($v_5256);
4  $v_5238 = array('encoding' => $v_5257);
5  try{
6  $v_5254 = '5z3Ovs6/z4DPgc+Dz4TPhc+Gz4fPiM+J';
7  $v_5255 = base64_decode($v_5254);
8  $v_5238 = array('encoding' => $v_5257);
9  $v_5245 = new SoapServer($v_5255,$v_5238);}
10 catch (Throwable $e){}
```

Ablated Variant	Component	Cov.	P-Value (F)	P-Value (V)
No Data Flow	Full	36.1	0.0021	0.0469
No Control Flow	-	37.3	0.0021	-
SEQFUZZ	-	**41.2**	-	-
No Data Flow	Zend	44.0	0.0021	0.0340
No Control Flow	-	45.1	0.0021	-
SEQFUZZ	-	**46.7**	-	-
No Data Flow	Main Functions	27.9	0.0400	0.2237
No Control Flow	-	28.3	0.0262	-
SEQFUZZ	-	**29.7**	-	-
No Data Flow	Modules	31.4	0.0021	0.0469
No Control Flow	-	33.0	0.0407	-
SEQFUZZ	-	**36.5**	-	-

TABLE 3: Ablation Study Results

Vulnerability Analysis: Memory Leak in SOAP. PHP's Simple Object Access Protocol (SOAP) enables the creation and usage of web services. Illustrated by Listing 7, a memory leak was present due to persistent memory not being freed. SEQFUZZ discovers this vulnerability by declaring `SoapServer` (Line 9) with unexpected arguments. The PHP development team noted, "*The root cause is that an error bailout happens if the wsdl filename is not found, which skips the free_obj hooks which means that the persistent memory from libxml is not freed here.*"[4]

Answer to RQ1: Analysis shows that SEQFUZZ is capable of uncovering a diverse set of critical bugs affecting a wide array of PHP's components.

6.2.2. Ablation Study. Figure 5 presents the 24-hour code coverage trends for SEQFUZZ and its ablated variants across the PHP interpreter and its primary subsystems. Solid lines indicate the average (arithmetic mean) coverage, while shaded regions represent the range across trials. All comparisons are statistically significant except in the *main functions*, where the differences between the two ablated variants are not significant (Table 3). The p-values for the ablated variants against the full SEQFUZZ are listed in the fourth column, and the p-values for the variants against each other are in the fifth column.

Full System. The complete SEQFUZZ configuration achieves the highest coverage across the full PHP interpreter at 41.2%. Ablation of data flow mutations reduces coverage to 36.1%, while removal of control flow mutations yields 37.3%. These results indicate that both mutation classes positively impact coverage, with data flow mutations contributing slightly more to overall interpreter coverage.

Zend Engine. Coverage within the Zend engine is the highest for the full SEQFUZZ, followed by the no-control-flow and no-data-flow variants. The Zend engine is responsible for execution, optimization, and memory management. While the exclusion of data flow mutations had a larger impact on performance, control flow mutations introduce more complex branching and looping structures, which contribute to the full system achieving the highest performance.

Main Functions. SEQFUZZ attains the highest coverage in the main functions. No statistically significant difference is observed between the control and data flow ablations. Because the main functions perform tasks such as setting up the runtime and parsing arguments, control and data flow mutations do not greatly increase coverage.

Modules. In PHP modules, the full system again achieves the highest coverage. Coverage drops more when data flow mutations are removed compared to when control flow mutations are ablated. This indicates that varying input values is especially important for testing PHP's extension modules.

Answer to RQ2: Results indicate that combining both control and data flow mutation types drastically improves system performance.

Fuzzer	FlowFusion [7]	PolyGlot [9]	Domato [8]	SEQFUZZ
Coverage	36.7%	24.8%	17.2%	**41.2%**
P-Value	0.0011	0.0006	0.0006	-
Bugs	1.14	0.14	0	**3.2**
P-Value	0.0023	0.0007	0.0006	-
Validity	2.4%	40.9%	1.5%	**51.0%**
P-Value	0.0041	0.0262	0.0041	-
Throughput	46.0k	1,860k	25.0k	462k
P-Value	0.0005	0.0011	0.0012	-

TABLE 4: Comparison against State-of-the-art

6.2.3. Coverage and Bug Discovery. Table 4 displays the average performance of SEQFUZZ, FlowFusion, PolyGlot, and Domato over seven independent trials with Figure 6 illustrating the average coverage trends. SEQFUZZ achieved the highest average coverage at **41.2%**, outperforming FlowFusion (36.7%), PolyGlot (24.8%), and Domato (17.2%). In terms of bug discovery, SEQFUZZ averaged **3.2 unique bugs per trial**, compared to FlowFusion's 1.14, PolyGlot's 0.14, and Domato, which failed to discover any bugs. These differences highlight limitations of each baseline. **Domato**, as a grammar-based fuzzer, is constrained by the expressiveness and correctness of its PHP grammar. Crafting a grammar that fully captures interpreter functionality is inherently difficult. Consequently, Domato achieves limited coverage and fails to discover bugs in the interpreter. **PolyGlot** improves upon Domato by leveraging a generalized BNF-based frame-

4. The presented vulnerabilities have been fixed by the PHP developers. All detected bugs were responsibly disclosed (Ethical Considerations).

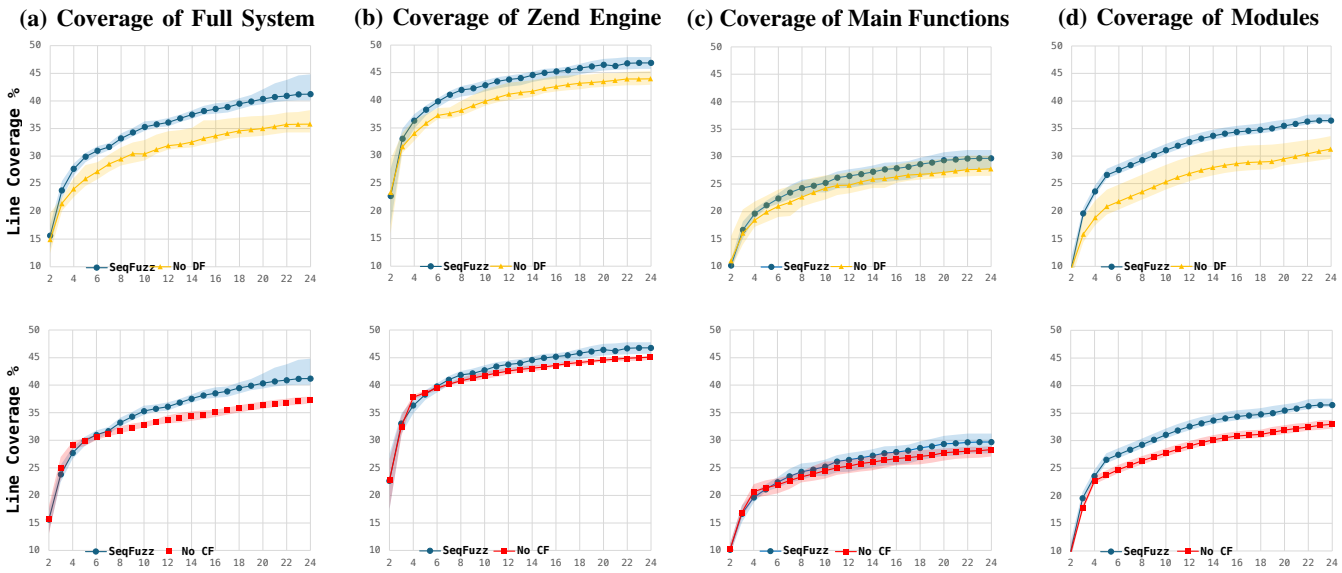


Figure 5: Coverage Comparison of Ablated Variants vs SEQFUZZ.

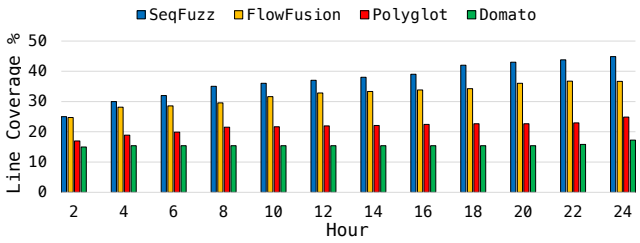


Figure 6: Coverage Comparison against State of the Art

work capable of producing syntactically valid programs. Yet, its lack of PHP-specific knowledge prevents it from generating seeds that exercise nuanced interpreter behaviors. While it discovers a small number of bugs, its effectiveness is inferior to FlowFusion and SEQFUZZ. **FlowFusion** employs lightweight data flow analysis to combine and mutate existing PHP scripts, enabling reasonable coverage and bug discovery. However, by focusing solely on data-flow recombination, it fails to explore deeper control-flow interactions or complex interpreter behaviors.

In contrast, SEQFUZZ's sequence-based IR explicitly encodes both data and control flows. Mutation operators enable the fuzzer to systematically explore a broader range of paths, resulting in higher coverage and the discovery of more unique bugs. These results illustrate that combining data flow and control flow mutations with a robust representation provides a substantial advantage over existing state-of-the-art fuzzers.

Answer to RQ3: SEQFUZZ's performance exceeds state-of-the-art baselines in both coverage and bug finding ability. The expressiveness of SEQIR enables SEQFUZZ to perform a wide range of mutation operators which contribute to its greater effectiveness.

6.2.4. Seed Validity. We evaluate SEQFUZZ's ability to generate valid PHP programs over seven independent trials (third row of Table 4). On average, SEQFUZZ produced **510.3 valid seeds per 1000** generated seeds, corresponding to a **51.0% validity rate**. In contrast, the official PHP fuzzer, FlowFusion, achieved only 24.3 valid seeds (2.4%), PolyGlot produced 409.0 valid seeds (40.9%), and Domato produced 14.9 valid seeds (1.5%).

SEQFUZZ's invalid seeds are primarily caused by fatal type mismatches. PHP is dynamically typed. Rather than enforcing typing constraints during mutation, SEQFUZZ allows arguments of arbitrary types which tests the interpreter's ability to handle type coercions and unexpected function inputs.

High seed validity is critical for fuzzing, as it ensures that the fuzzer exercises the interpreter's behaviors. However, validity alone is insufficient: a fuzzer could trivially achieve high validity by generating shallow, valid programs that lack meaningful operations. For example, a naive fuzzer producing seeds like `$a = 1; $b = 2;` will have a validity rate of 100% [11], yet such seeds are unlikely to trigger bugs.

SEQFUZZ overcomes this limitation by balancing validity and semantic diversity. Mutations on SEQIR preserve the semantic dependencies of the original programs while introducing meaningful variations. This enables SEQFUZZ to achieve high code coverage and discover more unique bugs than state-of-the-art fuzzers.

Answer to RQ4: Findings reveal that SEQFUZZ greatly outperforms state-of-the-art fuzzers in generated seed validity. SEQIR's flexible nature enables mutations while preserving program integrity.

6.2.5. System Throughput. Shown in Table 4, over seven independent 24-hour trials, SEQFUZZ averaged **462k seed executions**. This marks an 18.5× improvement over grammar-based Domato and a 10.0× improvement over PHP's official FlowFusion, but falls short of PolyGlot's throughput. The strong performance of SEQFUZZ suggests that SEQIR does not introduce substantial overhead.

Whereas FlowFusion operates using regular expressions and must continually re-analyze PHP seeds, SEQFUZZ stores seeds as SEQIR, allowing immediate manipulation without re-translation from PHP to SEQIR.

While high throughput is critical for fuzzing, high-quality seed production matters more, as complex seeds are required to uncover vulnerabilities. Although PolyGlot achieved higher throughput, SEQFUZZ outperformed it in seed validity, coverage, and importantly, bug discovery.

Answer to RQ5: SEQFUZZ has an 18.5× performance improvement over Domato and a 10.0× improvement over FlowFusion, indicating that SEQIR is well-suited for efficient fuzzing.

7. Discussion

Feedback. SEQFUZZ, like FlowFusion and other popular fuzzers [11], [7], [22], follows a black-box approach. A gray-box approach could improve fuzzing effectiveness, where mutation is guided by code coverage. Furthermore, other works suggest that a probabilistic language model can be effective for improving performance [27], [28]. However, such features are not central to this paper. Our key contribution is the introduction of an expressive, robust IR which allows for the efficient mutation and lifting of PHP programs.

Dependencies. SEQFUZZ depends on the presence of a high quality input corpus to thoroughly test interpreter functions. Using the official PHP test suite benefits SEQFUZZ by providing a continuously updated input corpus which reflects interpreter changes.

Extending to other Languages. SEQFUZZ's ability to generate complex seeds at high volume can be attributed to the expressiveness and robustness of SEQIR. The low constraints imposed on SEQIR enable mutation operators to manipulate sequences without time-consuming constraint analysis. Our upcoming research direction involves the extension of SEQFUZZ to other languages. Given the availability of an AST translator, other dynamic programming languages (e.g., JavaScript) can have their source code translated to SEQIR and thus be fuzzed by SEQFUZZ.

8. Related Work

Fuzzing. Fuzzing is a powerful strategy for uncovering critical vulnerabilities across various domains including compilers [29], [30], [31], SMT solvers [32], databases [33], and kernels [34], [35], [36]. In the domain of interpreter fuzzing, many systems and techniques have been created to uncover vulnerabilities deep within the target interpreter.

Google's Atheris [37] uses coverage-guided mutations to fuzz CPython extensions. For JavaScript, LangFuzz [22] combines grammar-based fuzzing and mutation-based fuzzing to recombine inputs into valid seeds. DIE uses aspect-preserving mutation operators to manipulate ASTs [17]. Systems like Montage [38], [39] use machine learning techniques to identify bug patterns and create similar inputs.

PHP has a weaker security tooling infrastructure. Tools like Domato [8], Gramatron [40], and NAUTILUS [41] fuzz PHP using grammars; yet, creating a comprehensive grammar requires significant manual effort, and the grammar must reflect interpreter updates. PolyGlot [9] is a BNF-based fuzzer which uses annotations to generate seeds for multiple languages. Recently, FlowFusion [7] uses lightweight program analysis to merge the data flows of two seeds.

SEQFUZZ addresses the gap in the PHP security infrastructure by introducing a method to stress all aspects of the interpreter's handling of control and data flow behaviors.

IR-based Fuzzing. IRs have been proven valuable for testing and fuzzing dynamic languages. For instance, Fuzzilli [10] introduces FuzzIL, a bytecode-level IR for JavaScript that models the semantics of JIT-executed code. By performing mutations at the IR level, Fuzzilli preserves syntactic validity while exercising deep JIT optimization logic, leading to the discovery of numerous high-impact vulnerabilities. FlowIR [12] extends this idea by translating JavaScript seeds into a graph-based IR that exposes control and data flow relationships, thereby enabling structurally aware mutations that outperform AST-based approaches. CodeAlchemist [11] decomposes JavaScript programs into statement-level fragments labeled with assembly constraints, allowing fragments to be recombined into syntactically valid programs.

We design SEQIR, a fuzzing-oriented IR for PHP. Unlike graph-based IRs or flat sequence-based approaches, SEQIR represents programs as hierarchical sequences of dependency-annotated nodes, explicitly encoding both control flow and data flow. This structure captures variable scoping, avoids graph isomorphism during mutation, and enables semantics-aware data and control flow transformations.

9. Conclusion

We presented SEQFUZZ, a novel fuzzing framework for the PHP interpreter built on SEQIR, a sequence-based intermediate representation that explicitly encodes control flow and data flow information. Our evaluation demonstrates that SEQFUZZ significantly outperforms state-of-the-art fuzzers, achieving higher code coverage, discovering 65 confirmed bugs including 52 zero-days, and producing seeds at a substantially higher validity rate. Ablation studies further show that both control flow and data flow mutations are crucial to its performance.

Ethics Considerations

We carefully considered the ethical implications of this work through every step of development, testing, and deployment. We identify relevant stakeholders, the impact of this work, and mitigatory steps taken to ensure ethical research.

Stakeholder Considerations. We identified four categories of stakeholders during the development and testing of SEQFUZZ: (1) *PHP interpreter developers*: who are responsible for the creation and maintenance of the official PHP interpreter, (2) *PHP application developers*: who use PHP for application development, (3) *end users*: who rely on PHP-powered services, and (4) *security researchers*: who develop tools for automated program testing and verification.

Impact. This work benefits the PHP ecosystem and its users, enabling PHP developers to find and patch bugs before end users are impacted. It provides a new state-of-the-art tool to better assess both the security and functionality of the official PHP interpreter.

Mitigations. To prevent ethical risks, we took the following steps:

- No Harmful Experiments. All fuzzing campaigns described in this paper were performed on our own isolated virtual machines and test servers with proper permissions. We did not fuzz any production systems, third-party websites, or services without explicit permission.
- Responsible Disclosure. During the development and evaluation of SEQFUZZ, we discovered numerous previously unknown issues in the PHP interpreter. All discovered vulnerabilities were responsibly disclosed to the PHP development team in accordance with their established disclosure procedure. We cooperatively worked with the PHP development team to triage reported vulnerabilities. We have withheld detailed proof-of-concept triggers for unpatched vulnerabilities from this paper.
- Data and Reproducibility. All datasets were derived from public sources. We did not collect or use any private data or sensitive real-world inputs during our research.

Throughout this work's development, we strictly adhered to PHP's bug reporting guidelines. Acknowledging the ethical implications of fuzzing research, we took proper steps to mitigate ethical risks. We conclude that the benefits of this research (enhanced PHP security, new state-of-the-art security techniques) outweigh the mitigated risks. Thus, the conducting and publishing of this research is ethically justified and in the public interest.

References

[1] Php manual. https://www.php.net/manual/en/index.php, 2025.

[2] W3Techs. Usage statistics and market share of php for websites. https://w3techs.com/technologies/details/pl-php. Accessed: November 21, 2024.

[3] CVE Details. Php vulnerabilities - php. https://www.cvedetails.com/product/128/PHP-PHP.html, 2025. Accessed: 2025-08-04.

[4] Ahmed Lekssays. Null pointer dereference in php soap extension via large xml namespace prefix. GitHub Security Advisory GHSA-453j-q27h-5p8x, 2025.

[5] TimWolla. Header parser of 'http' stream wrapper does not handle folded headers. GitHub Security Advisory GHSA-v8xr-gpvj-cx9g, 2025.

[6] bukka. Php cgi parameter injection vulnerability (cve-2024-4577 bypass). GitHub Security Advisory GHSA-p99j-rfp4-xqvq, 2024.

[7] Yuancheng Jiang, Chuqi Zhang, Bonan Ruan, Jiahao Liu, Manuel Rigger, Roland H. C. Yap, and Zhenkai Liang. Fuzzing the PHP interpreter via dataflow fusion. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6143–6158, Seattle, WA, August 2025. USENIX Association.

[8] Ivan Fratric. The great dom fuzz-off of 2017, 2017.

[9] Yongheng Chen, Rui Zhong, Hong Hu, Hangfan Zhang, Yupeng Yang, Dinghao Wu, and Wenke Lee. One engine to fuzz'em all: Generic language processor testing with semantic validation. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 642–658. IEEE, 2021.

[10] Samuel Groß, Simon Koch, Lukas Bernhard, Thorsten Holz, and Martin Johns. Fuzzilli: Fuzzing for javascript jit compiler vulnerabilities. In *NDSS*, 2023.

[11] HyungSeok Han, DongHyeon Oh, and Sang Kil Cha. CodeAlchemist: Semantics-aware code generation to find vulnerabilities in javascript engines. 2019.

[12] Haoran Xu, Zhiyuan Jiang, Yongjun Wang, Shuhui Fan, Shenglin Xu, Peidai Xie, Shaojing Fu, and Mathias Payer. Fuzzing javascript engines with a graph-based ir. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3734–3748, 2024.

[13] Jin Huang, Yu Li, Junjie Zhang, and Rui Dai. Uchecker: Automatically detecting php-based unrestricted file upload vulnerabilities. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 581–592. IEEE, August 2019. Publisher Copyright: © 2019 IEEE.; 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2019 ; Conference date: 24-06-2019 Through 27-06-2019.

[14] Dmitry Stogov. IR: Lightweight jit compilation framework. https://github.com/dstogov/ir, 2025. accessed: 2025-09-13.

[15] Cliff Click and Michael Paleczny. A simple graph-based intermediate representation. In *Papers from the 1995 ACM SIGPLAN Workshop on Intermediate Representations*, IR '95, page 35–49, New York, NY, USA, 1995. Association for Computing Machinery.

[16] Nikita Popov. php-ast: Php abstract syntax tree extension. https://github.com/nikic/php-ast, August 2025. version 1.1.3.

[17] Soyeon Park, Wen Xu, Insu Yun, Daehee Jang, and Taesoo Kim. Fuzzing javascript engines with aspect-preserving mutation. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1629–1642, 2020.

[18] PHP Group. Php test suite. https://github.com/php/php-src/tree/master/tests, 2025. Accessed: 2025-02-08.

[19] Guido Van Rossum. *The Python Library Reference, release 3.8.2*. Python Software Foundation, 2020.

[20] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. Magma: A ground-truth fuzzing benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 4(3):1–29, November 2020.

[21] Jonathan Metzman, László Szekeres, Laurent Simon, Read Sprabery, and Abhishek Arya. Fuzzbench: an open fuzzer benchmarking platform and service, 2021.

[22] Christian Holler, Kim Herzig, and Andreas Zeller. Fuzzing with code fragments. In *21st USENIX Security Symposium (USENIX Security 12)*, pages 445–458, Bellevue, WA, August 2012. USENIX Association.

[23] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing, 2018.

[24] Gcovr: Generate code coverage reports with gcc/gcov. https://gcovr.com/en/stable/, 2025. Accessed: 2025-12-16.

[25] Moritz Schloegel, Nils Bars, Nico Schiller, Lukas Bernhard, Tobias Scharnowski, Addison Crump, Arash Ale-Ebrahim, Nicolai Bissantz, Marius Muench, and Thorsten Holz. Sok: Prudent evaluation practices for fuzzing. In *IEEE Symposium on Security and Privacy (SP)*, 2024.

[26] MITRE Corporation. Common Weakness Enumeration (CWE). https://cwe.mitre.org/, 2025.

[27] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Skyfire: Data-driven seed generation for fuzzing. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 579–594, 2017.

[28] X. Jin, C. Dumitrescu, and D. Marinov. Treefuzz: Fuzzing with syntax trees. Technical Report TR-2016-11, Software Lab, Nov 2016.

[29] Karine Even-Mendoza, Arindam Sharma, Alastair F Donaldson, and Cristian Cadar. Grayc: Greybox fuzzing of compilers and analysers for c. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1219–1231, 2023.

[30] Xiaoting Li, Xiao Liu, Lingwei Chen, Rupesh Prajapati, and Dinghao Wu. Fuzzboost: Reinforcement compiler fuzzing. In *International Conference on Information and Communications Security*, pages 359–375. Springer, 2022.

[31] Hyungseok Kim, Soomin Kim, Jungwoo Lee, and Sang Kil Cha. Asfuzzer: Differential testing of assemblers with error-driven grammar inference. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*, ISSTA '24, pages 1099–1111, Vienna, Austria, September 2024. Association for Computing Machinery.

[32] Muhammad Numair Mansur, Maria Christakis, Valentin Wüstholz, and Fuyuan Zhang. Detecting critical bugs in smt solvers using blackbox mutational fuzzing. In *Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, pages 701–712, 2020.

[33] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. {DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4949–4965, 2023.

[34] Marius Fleischer, Dipanjan Das, Priyanka Bose, Weiheng Bai, Kangjie Lu, Mathias Payer, Christopher Kruegel, and Giovanni Vigna. Actor: action-guided kernel fuzzing. In *Proceedings of the 32nd USENIX Conference on Security Symposium*, SEC '23, USA, 2023. USENIX Association.

[35] Hao Sun, Yuheng Shen, Jianzhong Liu, Yiru Xu, and Yu Jiang. KSG: Augmenting kernel fuzzing with system call specification generation. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 351–366, Carlsbad, CA, July 2022. USENIX Association.

[36] Jiacheng Xu, Xuhong Zhang, Shouling Ji, Yuan Tian, Binbin Zhao, Qinying Wang, Peng Cheng, and Jiming Chen. Mock: Optimizing kernel fuzzing mutation with context-aware dependency. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS 2024)*, San Diego, CA, USA, 2024. The Internet Society.

[37] Google Open Source Team. Atheris: A coverage-guided, native python fuzzer. Accessed: 2025-10-18.

[38] Suyoung Lee, HyungSeok Han, Sang Kil Cha, and Sooel Son. Montage: A neural network language {Model-Guided}{JavaScript} engine fuzzer. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2613–2630, 2020.

[39] Haoran Xu, Yongjun Wang, Zhiyuan Jiang, Shuhui Fan, Shaojing Fu, and Peidai Xie. Fuzzing javascript engines with a syntax-aware neural program model. *Computers & Security*, page 103947, 2024.

[40] Prashast Srivastava and Mathias Payer. Gramatron: effective grammar-aware fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2021, page 244–256, New York, NY, USA, 2021. Association for Computing Machinery.

[41] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. Nautilus: Fishing for deep bugs with grammars. In *NDSS*, 2019.